Rapport de projet de Programmation Fonctionnelle

Éditeur de texte avec Ocaml

THOUAN Simon

- Encadré par Julien Forest -

07/05/2025

Table des matières

1 Introduction **2**

2 Fonctionnement du programme **2**

2.1 Les types utilisés 2

2.2 Description du comportement des fonctions originales 2

2.3 Description des ajouts apportés aux fonctions 3

3 Problèmes rencontrés **4**

4 Limites du programme et conclusion **5**

5 Annexe **5**

1 Introduction

L'objectif de ce projet est de créer un programme Ocaml permettant la gestion d'un éditeur de texte simple.
On doit pouvoir naviguer dans le fichier, écrire des caractères et les supprimer.

2 Fonctionnement du programme

L'utilisation du Makefile fournit un fichier nommé test qui peut être exécuté dans un environnement linux. On dispose alors d'une fenêtre qui, si toutes les fonctions ont été correctement implémentées, répond à la spécification détaillée en introduction.

2.1 Les types utilisés

Nous utilisons ici uniquement les types fournis par l'énoncé, en particulier "line" et "buffer". Pour les déplacements, on utilise plutôt la première approche, à savoir deux fonctions opérant sur le type "ligne" (*move_right* et *move_left*) et deux autres fonctions opérant sur le type "buffer" (*move_up* et *move_down*).

2.2 Description du comportement des fonctions originales

- **get_current** : renvoie la valeur courante du zipper z *via* l'accès au champ current de z.
- **get_pos** : renvoie la position courante du zipper z *via* l'accès au champ pos de z.
- **fold_zipper** : effectue un fold sur le type des zipper. Son type est plus général que celui exigé par l'énoncé. Pour retrouver celui de l'énoncé, il faut respecter les préconditions et utiliser une fonction g de type ('b -> 'c -> 'c). En toute généralité, cette fonction peut gérer des fonctions de type ('b -> 'c -> 'd).
- **update_with** : application de f sur le champ current de z.
- **move_left** : effectue un déplacement vers la gauche de la ligne courante.
- **move_right** : effectue un déplacement vers la gauche de la ligne courante.
- **move_up** : effectue un déplacement vers le haut dans le fichier.
- **move_down** : effectue un déplacement vers le bas dans le fichier.

— **insert_char** : ajoute le char à la position courante en avançant la position de 1.
— **do_suppr** : supprime le caractère courant.
— **do_backspace** : supprime le caractère précédent le caractère courant.
— **create_newline** : insère un saut de ligne à la position courante.

2.3 Description des ajouts apportés aux fonctions

Nous allons voir ici le comportement des fonctions, dans leur implémentation simple décrite précédemment (on pourra retrouver une description similaire de ce comportement de base en regardant notamment le champ "@ensures" dans les commentaires du code des fonctions concernés). Sont concernés par des modifications uniquement les fonctions qui sont mentionnées dans cette partie.
Ensuite, nous décrirons (et illustrerons en annexe) le comportement amélioré (qui est décrit par le champ "@update" qui apparaît, le cas échéant, lui aussi dans les commentaires du code des fonctions concernés).

(a) Description du comportement des fonctions avant l'implémentation du champ "@update"
   — **move_left** : lorsque le curseur est au début d'une ligne, ne fait rien. Sinon, se déplace d'un caractère sur la gauche.
   — **move_right** : lorsque le curseur est à la fin d'une ligne, ne fait rien. Sinon, se déplace d'un caractère sur la droite.
   — **move_up** : lorsque le curseur est en haut du fichier, ne fait rien. Sinon, déplace le curseur à la ligne précédente, en le plaçant à la position où il était quand il a quitté la ligne cible.
   — **move_down** : lorsque le curseur est en bas du fichier, ne fait rien. Sinon, déplace le curseur à la ligne suivante, en le plaçant à la position où il était quand il a quitté la ligne cible.
   — **do_suppr** : si le curseur est en fin de ligne, il ne se passe rien. Sinon, supprime le caractère courant.
   — **do_backspace** : si le curseur est en début de ligne, il ne se passe rien, sinon, supprime le caractère précédent le caractère courant.

(b) Description du comportement des fonctions après l'implémentation du champ "@update"
   — **move_left** : lorsque le curseur est au début d'une ligne, se déplace à la fin de la ligne précédente. Si le curseur est au début de la première ligne du fichier, ne fait rien.
   — **move_right** : lorsque le curseur est à la fin d'une ligne, se déplace

au début de la ligne suivante. Si le curseur est à la fin de la dernière ligne du fichier, crée une nouvelle ligne vide.

- **move_up** : lorsque le curseur est en haut du fichier, déplace le curseur au début de la ligne courante. Sinon, déplace le curseur à la ligne précédente, en le plaçant à la même position que la ligne d'où il vient.
- **move_down** : lorsque le curseur est en bas du fichier, renvoie le curseur à la fin de la ligne courante. Sinon, déplace le curseur à la ligne suivante, en le plaçant à la même position que la ligne d'où il vient.
- **do_suppr** : si le curseur est en fin de ligne, supprime la ligne suivante et ramène son contenu à la ligne courante, après le curseur, sauf si la ligne courante est la dernière ligne, auquel cas il ne se passe rien.
- **do_backspace** : si le curseur est en début de ligne, ramène le curseur et le contenu de la ligne courante à la fin de la ligne précédente, sauf si la ligne courante est la première ligne, auquel cas il ne se passe rien.
- **create_newline** : insère un saut de ligne à la position courante. Une nouvelle ligne vide est créée si le saut de ligne est fait à la fin de la ligne. Sinon, le texte après le curseur est reporté sur la nouvelle ligne.

3 Problèmes rencontrés

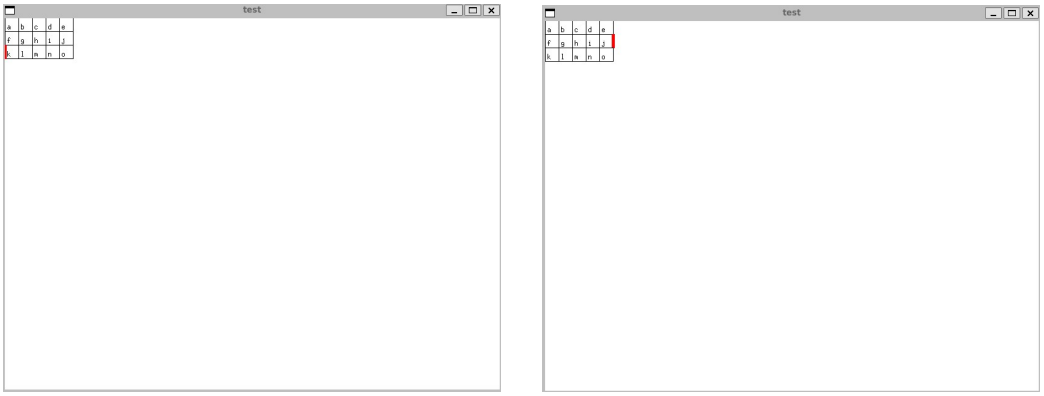
- Pas très important mais qui m'a fait perdre un peu de temps : l'ajout de " ;" inutile après l'implémentation des fonctions demandées. J'ai mis un peu de temps à comprendre le problème car la compilation me donnait seulement le message "syntax error" pour la ligne "let w_open = [...]"
- Pour que les modifications sur *do_backspace* et *do_suppr* soient compatibles avec les fonctions *move_up* et *move_down*, le décompte précis du champ "pos" des zipper de ligne modifiés dans les fonctions de suppressions de caractère doit être réalisé. La fonction *last_cur_pos*, lorsqu'elle est bien utilisée, répond à cette problématique.

4 Limites du programme et conclusion

Ce programme, même s'il gère une bonne partie des déplacements utiles et classiques, ainsi que l'insertion et la suppression de caractères, ne permet pas, et de loin, de gérer autant de choses qu'un bon gestionnaire de fichier comme vim par exemple. Des pistes d'amélioration seraient l'introduction des commandes classiques telles que le recherche, la copie de texte, le fait de pouvoir sauvegarder les fichiers tapés, ... etc.

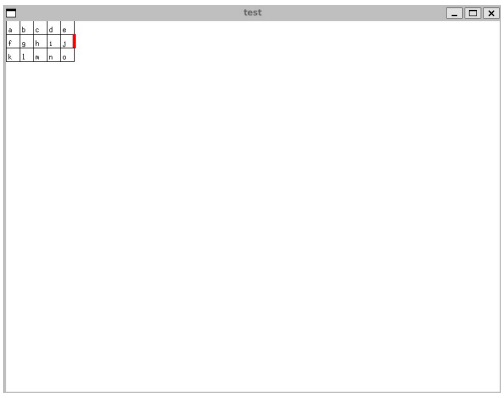
5 Annexe

Les figures ci-dessous illustrent les modifications apportées aux fonctions qu'on a voulu améliorer. Ces modifications sont décrites dans la partie 2.3(b).

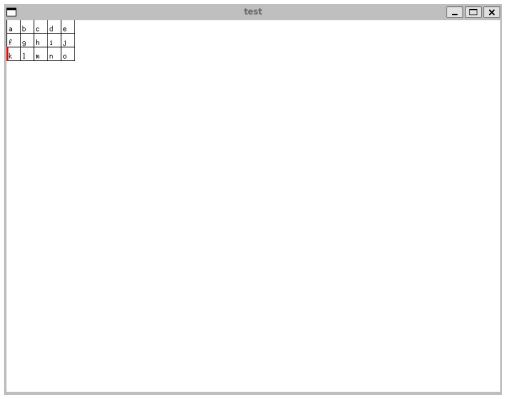


(a) Avant exécution de move_left (b) Après exécution de move_left

FIGURE 1 – Comparaison avant et après l'exécution de move_left

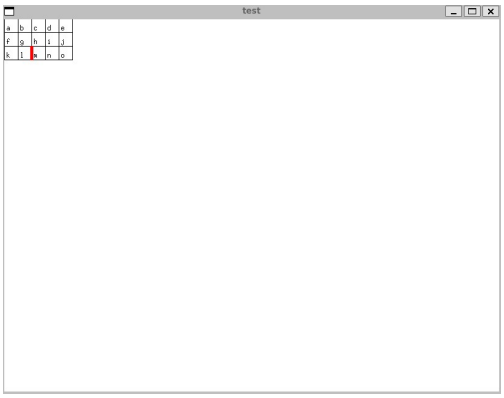


(a) Avant exécution de move_right

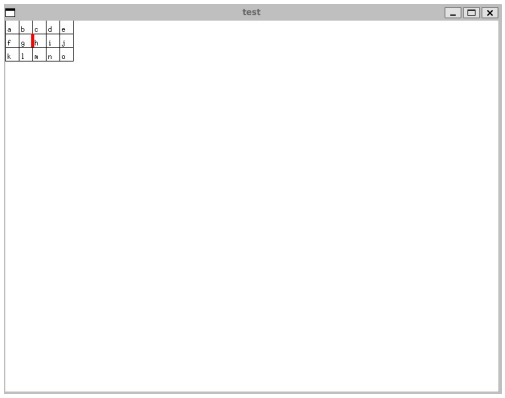


(b) Après exécution de move_right

FIGURE 2 – Comparaison avant et après l'exécution de move_right

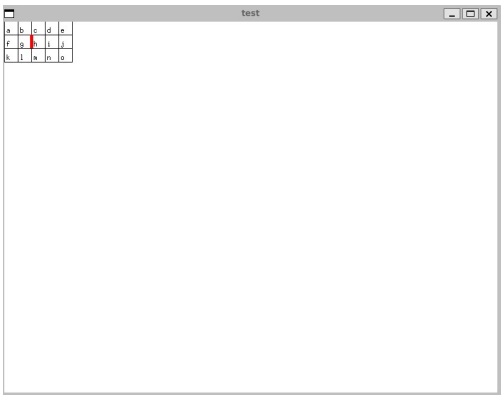


(a) Avant exécution de move_up

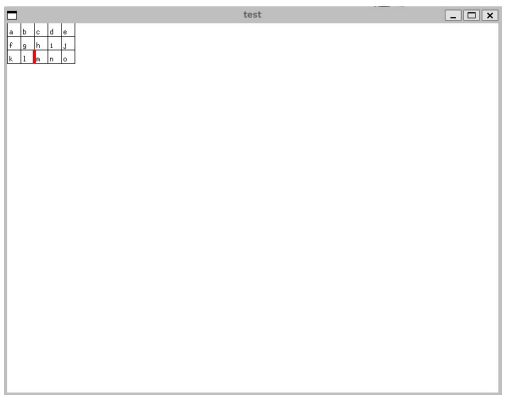


(b) Après exécution de move_up

FIGURE 3 – Comparaison avant et après l'exécution de move_up

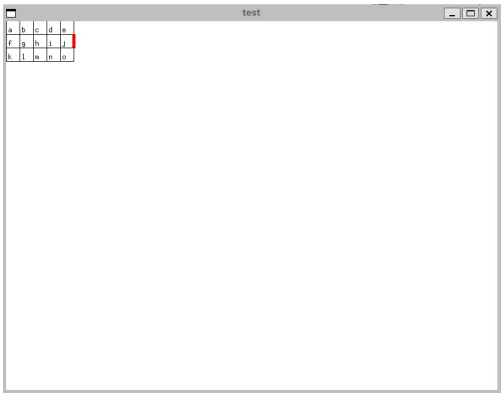


(a) Avant exécution de move_down

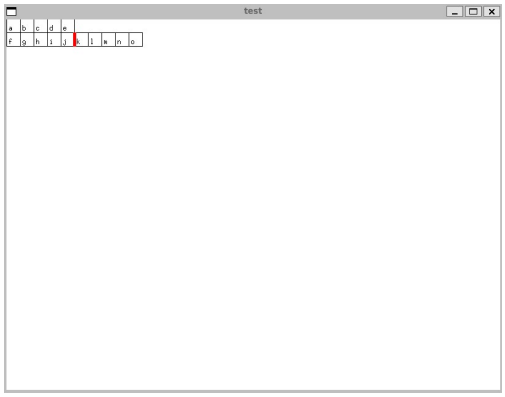


(b) Après exécution de move_down

FIGURE 4 – Comparaison avant et après l'exécution de move_down

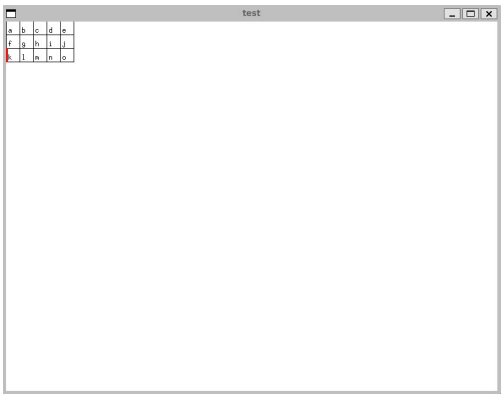


(a) Avant exécution de do_suppr



(b) Après exécution de do_suppr

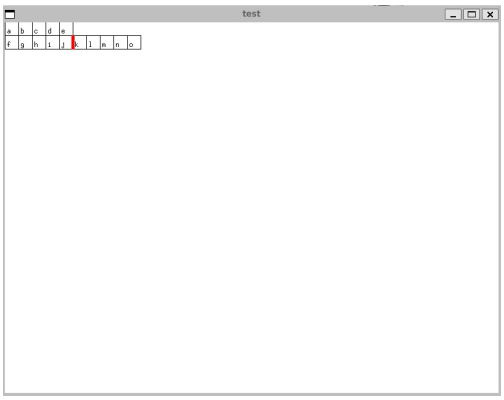
FIGURE 5 – Comparaison avant et après l'exécution de do_suppr



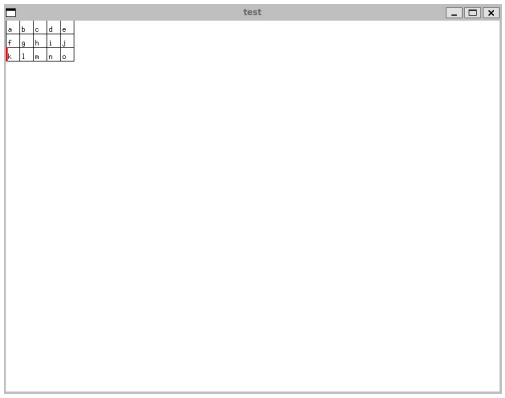
(a) Avant exécution de do_backspace

(b) Après exécution de do_backspace

FIGURE 6 – Comparaison avant et après l'exécution de do_backspace



(a) Avant exécution de create_newline



(b) Après exécution de create_newline

FIGURE 7 – Comparaison avant et après l'exécution de create_newline